

Сценарий защиты — Ecosystem Simulation

Yermek Aubayev (551098)

2026-09-14

Перевод `defense_script.md` на русский — для понимания и репетиции. Код, имена классов/методов, номера строк и цитаты консольного вывода оставлены как есть (это факты, не текст для перевода). Блоки “Сказать:” переведены на русский, чтобы был понятен смысл аргумента — но сама защита, скорее всего, идёт на английском (весь проект, отчёт и rebuttal — на английском), поэтому для дословной формулировки держи под рукой английский оригинал: [defense_script.md](#) / [defense_script.pdf](#). Компаньон — [theory.md](#) (там тоже пока английский; скажи, если нужен перевод и его).

0. Как этим пользоваться

Запоминай идеи, а не фразы. Чтение сценария по бумажке на устной защите выглядит плохо и не переживёт вопрос, заданный не так, как ты репетировал. Если попросят что-то дописать в коде прямо на месте — рецепты в §5 сверены построчно с реальным исходным кодом на 2026-09-14 (Scavenger.java даже скомпилирован против `target/classes`) — открывай настоящие файлы и следуй тому же паттерну, а не набирай по памяти под давлением.

Каждый пункт с пометкой “△ уже поднималось” — из `docs/report/rebuttal.md`, четыре конкретных замечания, которые профессор уже делал по этому же проекту. Экзаменатор, поймавший что-то один раз, очень часто переспрашивает это же снова — проверить, что было реально понято, а не просто исправлено кем-то другим. §3 — самый ценный раздел, знать нужно твёрдо.

1. Вступительное слово (30–45 секунд)

Сказать что-то близкое к этому (на английском — рабочий язык проекта):

“This project simulates predators, herbivores, and plants interacting directly on a shared grid — no central controller decides outcomes, each entity queries the world and acts on its own. It was built around two technical requirements at once: first, a genuinely discrete-event simulation engine — not a fixed-timestep loop that just calls itself one — and second, a runtime rule language that lets you author new ecosystem behavior, like a compound temperature rule, without touching or recompiling any Java source. Around those two pillars, the project demonstrates every core OOP principle with examples the system actually depends on to function, not ones added to satisfy a checklist.”

Смысл по-русски: проект моделирует хищников, травоядных и растения, взаимодействующих напрямую на общей сетке — никакой центральный контроллер не решает исходы, каждая сущность сама опрашивает мир и действует. Построен вокруг двух технических требований

сразу: во-первых, по-настоящему дискретно-событийный движок — не цикл с фиксированным шагом, который просто называет себя так, — во-вторых, язык правил времени выполнения, позволяющий сочинять новое поведение экосистемы (например, составное правило про температуру) без изменения и перекомпиляции Java-кода. Вокруг этих двух столпов проект демонстрирует каждый ключевой принцип ООП на примерах, от которых система реально зависит, а не добавленных для галочки.

Последняя фраза — это утверждение, которое нужно быть готовым подтвердить в любом месте, куда укажет экзаменатор, а не только в отрепетированных разделах.

2. Экскурсия по архитектуре — что показывать, в каком порядке

1. **Иерархия сущностей** (доска или `class_diagram.puml`): `Entity` → `Organism` → `Animal` → `{Predator, Herbivore}`, `Plant` отдельной веткой от `Organism`. Причину, почему `Plant` не под `Animal`, проговорить самому, не дожидаясь вопроса (theory.md A.3).
2. **Цикл FEL**: открыть `SimulationEngine.java`, метод `advanceTo()` (строка 128). Прочитать цикл вслух: достаём ближайшее по времени событие, принимаем *его собственное* время как текущее, исполняем, повторяем. Явно противопоставить обычному `for`-циклу по всем сущностям — это как раз то, чего здесь нет.
3. **Жизненный цикл одной сущности**: `EntityActivityEvent` → сущность действует → сама планирует своего “преемника”. Отметить, что именно поэтому цепочка мёртвой сущности просто останавливается — для неё больше никто не планирует продолжение.
4. **DSL правил, целиком**: открыть `config/rules.txt` или встроенный редактор правил, показать составное правило (`CrowdedForest | Plant | stat.plantPopulation > 100 OR env.foodLevel < 20 | growthRate *= 0.5`), затем открыть `RuleVocabulary.java` и показать вызовы `registerReadable` / `registerWritable`, которые стоят за токенами именно этого правила.
5. **Доказательство мультитипизации**: запустить программу, показать вывод в консоли до появления окна — `Main.demonstrateMultityping()` печатает пять объявленных типов для одного объекта, заканчивая `entityRef == animalRef: true`.
6. **Тесты**: упомянуть 107 тестов, 13 классов, и назвать конкретно два-три действительно показательных (детерминированный порядок FEL при равных временах, ленивая инвалидация, атомарный откат при сохранении файла правил), а не просто цифру.

3. Вопросы повышенного риска — уже поднимались раньше

3.1 «Инкапсуляция и сокрытие информации — разве не одно и то же?»

Не повторяй старую, уже исправленную ошибку — не описывай видимость полей под заголовком “Encapsulation”. **Сказать**: “Это разные оси. Инкапсуляция — это контроль доступа к состоянию *одного конкретного объекта*: `World.getEntities()` возвращает защитную копию, чтобы вызывающий код не мог мутировать внутренний список `World` в обход него. Сокрытие информации — это про то, свободна ли *внутренняя реализация иерархии* меняться позже:

каждое поле от `Entity` до `Predator` — `private`, а не `protected`, включая абстрактные базовые классы, проверено 3-шаговым методом.” Держи наготове номера строк `World.java:85` и `Entity.java:24` (theory.md A.1–A.2).

3.2 «Докажи, что поля реально `private`, а не просто есть геттеры»

Это самая острая версия уже заданного вопроса — будь готов, что профессор буквально откроет файл и посмотрит. Они действительно `private` (проверено напрямую, актуальный код): `Entity.id/x/y/alive/world/statistics/` `simulationEventListener/simulationTime/schedulingContext`, `Organism.energy/age/maxEnergy`, `Animal.speed/visionRange`, `Predator.attackPower`, `Herbivore.defensePower`. Если спросят, почему это важно на практике — не отделиваясь определением, скажи: “`private` вместе с `final` делают `Entity.id` константой, гарантированной **компилятором**, а не просто соглашением, которое можно случайно нарушить строкой вроде `this.id = ...` в будущем подклассе. `protected` позволял бы это, и в одном месте так и произошло, до исправления — `Predator.hunt()` читал унаследованное поле `visionRange` напрямую, вместо вызова `getVisionRange()`.”

3.3 «Докажи мультитипизацию из клиентской программы, а не из объявлений типов»

Сказать: “`Main.demonstrateMultityping()`, вызывается в самом начале `main()`. Создаётся один объект `Predator`, и та же самая ссылка присваивается пяти переменным пяти разных объявленных типов — `Entity`, `Organism`, `Animal`, `Movable`, `Reproducible` — затем через каждую вызывается специфичный для типа метод, и в конце печатается `entityRef == animalRef`, что даёт `true`, потому что сравнение ссылок сравнивает сам объект, а не объявленный тип переменной, которая на него указывает.” **Лучше всего — предложить запустить программу прямо там** и показать вывод вживую, а не просто рассказать о нём. Реальный вывод в консоли (это буквально то, что печатает программа — переводить нечего):

```
as Entity      -> id=1, alive=true
as Organism    -> energy=80
as Animal      -> speed=2.0
as Movable     -> getSpeed() via interface=2.0
as Reproducible -> Predator implements reproduce()
identity check -> entityRef == animalRef: true
```

3.4 «Обработка исключений у тебя — просто определения классов, обработки никакой»

Сказать: “Два реальных места, где исключение ловится и с ним что-то делают. Первое — `RuleEditorPanel.doReload()`: ловит `RuleParseException` при повторном разборе вручную отредактированного `config/rules.txt` и явно оставляет предыдущие правила в памяти, не давая повреждённому файлу дойти до работающего движка. Это независимо проверяется тестом, `RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules`, а не просто утверждается в тексте отчёта. Второе — стартовый путь `MainWindow` раньше ловил те же исключения, но только писал в `System.err` — что невидимо в упакованном Swing-приложении и по сути не является обработкой, раз видимое поведение программы вообще не менялось. Теперь оба места пишут в тот же экраный журнал событий, которым и так пользуется вся остальная симуляция.” Если надавят на вопрос, что вообще требуется от “обработки”:

остановить падение программы, оставить её в одном конкретном, чётко определённом состоянии, сообщить пользователю, что произошло. `Catch`, который только печатает в консоль, которую никто не видит, выполняет только первое условие из трёх.

4. Другие вероятные вопросы

Основаны на собственных заявленных границах отчёта — экзаменатор, читающий раздел “Known Limitations”, как правило, проверяет именно эти границы, раз автор сам явно обозначил их как спорные.

- **«Почему AST для правил не использует полиморфизм, как всё остальное здесь?»** → Осознанный контрпример (theory.md A.5.1): набор узлов AST фиксирован и закрыт грамматикой, поэтому исчерпывающий `switch`, проверяемый компилятором, над `sealed-record`-ами — более типобезопасный инструмент здесь, чем виртуальная диспетчеризация, для которой понадобилась бы пара `accept() / visit()` на каждом узле ради той же гарантии (и без проверки исчерпываемости компилятором до появления этого в Java).
- **«DSL для правил — тьюринг-полный?»** → Нет, намеренно нет: ни циклов, ни пользовательских функций, ни рекурсии, закрыто на уровне типов через `sealed AST`, а не просто по договорённости. Сказать это самому, не дожидаясь вопроса — это уже написано в §4 отчёта.
- **«Почему два интерфейса, `Movable` и `Reproducible`, а не один?»** → Interface Segregation: `Plant` — `Reproducible`, но не `Movable`; единый интерфейс заставил бы `Plant` реализовывать `move()`, для которого нет осмысленного тела.
- **«Почему `Math.random()`, а не сидированный `Random`?»** → Заявленное ограничение: точная воспроизводимость запуска от запуска к запуску не гарантируется; тесты, зависящие от исходов симуляции, используют усреднение по батчу или структурно-детерминированные проверки вместо точных значений одного прогона — именно по этой причине.
- **«Где именно модульность не идеальна?»** → Сказать самому: `Statistics.adjustSpeciesCounter` и цепочка `instanceof` в `EntityFactory.createOffspringNear` независимо друг от друга перечисляют три конкретных вида. Четвёртый вид потребует обновить оба места — осознанный компромисс (не вводить абстракцию-реестр, которую было бы сложнее читать новому человеку), а не недосмотр.
- **«В чём точная граница компиляции/времени выполнения у твоей заявки про кастомизацию ‘без перекомпиляции’?»** → Составление новой логики из уже зарегистрированных атрибутов/команд не требует ни строчки Java. Открытие по-настоящему нового атрибута или команды требует одного вызова `registerX(...)` и перекомпиляции. Говорить обе половины, не только выгодную.
- **«Что если два события попадают на один и тот же тик?»** → `SimulationEvent.compareTo` разрывает ничью по монотонно растущему `sequenceNumber` (`AtomicLong`), поэтому события на одном тике всегда выполняются в порядке создания — детерминированно, потому что сам по себе `PriorityQueue` этого не гарантирует.
- **«Что если сущность умирает уже после того, как на неё запланировано событие, но до того, как оно выполнится?»** → Ленивая инвалидация: в FEL нет активного поиска-и-отмены (`PriorityQueue` не умеет эффективно удалять произвольный элемент); вместо этого каждое

событие в начале своего `execute()` перепроверяет `isAlive()` / `isPendingRemoval()` и молча ничего не делает, если данные устарели. `pendingRemoval` выставляется синхронно в момент, когда удаление запрашивается, — до того, как выполнится `DeathEvent`, который реально переключает `alive`.

5. «А теперь допиши это в коде» — рецепты живого кодирования

Три уровня, от дешёвого к дорогому. Если времени мало — так и скажи, и сделай самый дешёвый целиком, а не дорогой наполовину.

5.1 Самое быстрое (~2 мин): новый атрибут в словаре правил

Добавить в `RuleVocabulary.registerDefaults()` (строка 194) — например, записываемый `visionRange` для `Animal`:

```
registerReadable("visionRange", Animal.class, ctx -> ((Animal)
    ctx.entity()).getVisionRange());
registerWritable("visionRange", Animal.class, (ctx, newValue) ->
    ((Animal) ctx.entity()).setVisionRange((int) clamp(newValue, 1, 30)));
```

Говорить во время набора: “Это и есть точка расширения Open/Closed — один вызов здесь, больше ничего не меняется. `Lexer`, `Parser`, `Rule`, `RuleEngine` не тронуты, и `RuleVocabularyValidationTest` / `ParserTest` уже доказывают этот паттерн, добавляя покрытие поверх словаря, ни разу не трогая эти файлы.” Затем показать, что это работает: правило вида `Cautious | Animal | age > 50 | visionRange += 5` должно теперь успешно парситься и проходить валидацию.

5.2 Средний уровень (~3–4 мин): новый тип дискретного события

Наследуемся от `SimulationEvent`, реализуем два абстрактных метода. Минимальный скелет (по форме совпадает с шестью существующими классами событий):

```

package com.ecosystem.simulation.events;

import com.ecosystem.simulation.entities.Entity;

public class MigrationEvent extends SimulationEvent {

    private final Entity entity;

    public MigrationEvent(int scheduledTime, Entity entity) {
        super(scheduledTime, entity);
        this.entity = entity;
    }

    @Override
    public void execute(SchedulingContext ctx) {
        if (!entity.isAlive() || entity.isPendingRemoval()) {
            return;    // ленивая инвалидация — тот же паттерн, что и у всех остальных
                        // событий
        }
        // ... здесь — фактическая мутация состояния, за которую отвечает это событие ...
    }

    @Override
    public String getDescription() {
        return entity + " migrated";
    }
}

```

Говорить во время набора: “Та же защита ленивой инвалидации, с которой начинается любое другое событие. `EventQueue` и `SimulationEngine.advanceTo()` не требуют ни единого изменения — они взаимодействуют с событиями только через абстрактный базовый тип `SimulationEvent`, а это и есть полиморфизм включения, делающий реальную работу, а не украшение.” Запланировать такое событие можно вызовом `ctx.schedule(new MigrationEvent(ctx.getClock() + 10, this))` из того места (события или сущности), которое должно его вызывать.

5.3 Полный рецепт (~8–10 мин): новый вид

Если профессор не назовёт конкретный вид — взять **Scavenger**: имя уже упомянуто в Javadoc-комментарии самого `Reproducible.java` как пример гипотетического нового вида, так что это согласуется с уже существующими комментариями в коде. Поведение намеренно минимальное (движение + расход энергии + старение + размножение, без охоты/выпаса), чтобы реально успевать набрать текст под давлением времени.

Шаг 1 — сам класс (`entities/Scavenger.java`), по форме как `Herbivore.java` минус логика выпаса:

```

package com.ecosystem.simulation.entities;

import com.ecosystem.simulation.events.ReproductionEvent;

public class Scavenger extends Animal implements Reproducible {

    private int reproductionCooldown;
    private int maxAge = 60;
    private int energyConsumptionPerTick = 1;
    private int reproductionCooldownPeriod = 15;
    private int populationCap = 15;

    public Scavenger(int x, int y, int energy, double speed) {
        super(x, y, energy, speed);
        this.reproductionCooldown = 0;
        setVisionRange(15);
    }

    @Override
    public void update() {
        if (!isAlive()) {
            return;
        }

        move();
        consumeEnergy(energyConsumptionPerTick);
        if (!isAlive() || isPendingRemoval()) {
            return;
        }

        increaseAge();
        if (getAge() >= maxAge) {
            scheduleRemoval("old age");
            return;
        }

        if (reproductionCooldown > 0) {
            reproductionCooldown--;
        }
        if (getEnergy() > getMaxEnergy() * 0.6 && reproductionCooldown == 0) {
            reproduce();
        }
    }

    public void setMaxAge(int maxAge) { this.maxAge = maxAge; }
    public void setEnergyConsumptionPerTick(int amount) { this.energyConsumptionPerTick = amount; }
    public void setReproductionCooldownPeriod(int period) { this.reproductionCooldownPeriod = period; }
    public void setPopulationCap(int cap) { this.populationCap = cap; }
    public int getPopulationCap() { return this.populationCap; }

    @Override
    public void reproduce() {
        if (getWorld() == null || getSchedulingContext() == null) {
            return;
        }
        if (getWorld().countAliveByType("Scavenger") >= populationCap) {
            reproductionCooldown = 10;
            return;
        }
        int reproductionCost = getEnergy() / 2;
        consumeEnergy(reproductionCost);
        if (!isAlive() || isPendingRemoval()) {

```

```

        return;
    }
    getSchedulingContext().schedule(new
    ReproductionEvent(getSchedulingContext().getClock(), this));
    this.reproductionCooldown = reproductionCooldownPeriod;
}
}

```

Проговаривать во время набора: “`extends Animal` — бесплатно получаем управление энергией/возрастом/скоростью/дальностью обзора, `implements Reproducible` — не `Movable` напрямую, потому что `Animal` его уже даёт. `move()` наследуется в готовом виде от `Animal`, так что я не переопределяю его, если не нужно другое движение.”

Шаг 2 — подключить в `EntityFactory` (строка 36):

```

public Scavenger createScavenger(int x, int y) {
    Scavenger s = new Scavenger(x, y, 60, 1.8); // литералы для демонстрации;
    wire(s); // конфиг-версия — см. шаг 5
    return s;
}

```

...и добавить одну ветку в цепочку `instanceof` метода `createOffspringNear` (строка 78):

```

if (parent instanceof Scavenger) {
    return createScavenger(newX, newY);
}

```

Шаг 3 — зарегистрировать тип цели в `RuleVocabulary.TARGET_TYPES` (строка 56):

```

private static final Map<String, Class<? extends Entity>> TARGET_TYPES = Map.of(
    "Entity", Entity.class,
    "Organism", Organism.class,
    "Animal", Animal.class,
    "Predator", Predator.class,
    "Herbivore", Herbivore.class,
    "Plant", Plant.class,
    "Scavenger", Scavenger.class // новое
);

```

Понадобится ещё один новый импорт в начале этого файла — `import com.ecosystem.simulation.entities.Scavenger;` (файл импортирует каждый класс сущности по отдельности, без wildcard). `EntityFactory.java` новый импорт не нужен — он уже живёт в пакете `entities`, там же, где `Scavenger`.

Сказать прямо — это настоящий, точный нюанс, а не ошибка, которую нужно прятать:

“Раздел про расширяемость в отчёте говорит, что новому виду нужен ‘один новый метод `EntityFactory.createX(...)` плюс конфиг, без изменений движка’. Это верно для FEL и механизма диспетчеризации событий. Но не вся история именно для этого шага — токен `targetType` в грамматике правил проверяется по фиксированной на момент компиляции карте, `TARGET_TYPES`, так что если я хочу, чтобы `rules.txt` мог сказать `Scavenger | ...`, в эту карту нужно добавить ещё одну запись. Мелочь, но точность здесь важнее, чем повторение упрощённой версии из отчёта.”

Шаг 4 — заспавнить несколько при старте, в `SimulationEngine.initialize()` (строка 81):

```
for (int i = 0; i < config.initialScavengers(); i++) {
    spawnInitial(entityFactory.createScavenger(randomPos(world.getWidth()),
        randomPos(world.getHeight())));
}
```

...плюс один геттер в `SimulationConfig` (тот же паттерн, что у `initialPredators()` / `initialHerbivores()` / `initialPlants()`):

```
public int initialScavengers() { return getInt("population.scavenger", 5); }
```

Сказать и это тоже — более острая версия заявки про расширяемость: “Здесь я бы слегка поспорил с формулировкой самого отчёта. ‘Движок не требует изменений’ — верно для того, как уже существующая сущность работает после появления — FEL и `advanceTo()` действительно не важно, какой тип вынули из очереди. Но это неверно для того, как вид *бутстрапится* впервые — без этого цикла `Scavenger` просто неоткуда взяться, кроме как через размножение уже существующего, а его пока нет. Это реальный, маленький пробел в упрощённой формулировке, и я лучше сам его назову, чем дождусь, пока его найдут.”

Шаг 5 — опционально, если есть время (полный паритет с существующими видами):

конфиг-управляемое создание вместо литералов из шага 2 (добавить геттеры `scavengerEnergy()` / `scavengerSpeed()` в `SimulationConfig`, тот же паттерн `getInt` / `getDouble`), и ветку в `Statistics.adjustSpeciesCounter` (строка 130), если нужно, чтобы счётчики рождений/смертей учитывали именно `scavenger`-ов отдельно. **Заметка:** популяционные ссылки в правилах (`stat.scavengerPopulation`, если такую добавить) заработали бы и без изменений в `Statistics` вообще — `stat.*` читаемые атрибуты вызывают `world.countAliveByType("Scavenger")` напрямую на `World`, который считает живые сущности обобщённо, по простому имени класса.

6. Если не знаешь ответа

- Переформулировать вопрос своими словами вслух — это даёт время подумать и подтверждает, что вопрос понят правильно.
- Рассуждать от известных принципов дизайна (Лисков, Interface Segregation, Open/Closed), а не гадать про конкретную строку кода, которую не помнишь точно.
- Если честный ответ — “это известное ограничение”, так и сказать, и указать на §4 отчёта — этот раздел существует именно для того, чтобы не оказаться врасплох из-за границы, которую сам же и обозначил.
- Никогда не выдумывать утверждение, чтобы заполнить пробел. Точное “надо бы перепроверить” — гораздо лучший ответ, чем уверенный неверный, и из него легко выйти; выдуманное утверждение, которое проверят тут же, — нет.

7. Чего не говорить — преувеличения, которых стоит избегать

- Не утверждать “ни у кого на GitHub такого нет” или что-то в этом духе, даже если тема всплывёт. Честная, защитимая версия — гораздо уже: среди той пары сравнимых студенческих/хобби-проектов, что реально были прочитаны, ни один не сочетал настоящий

FEL с DSL правил времени выполнения — это не то же самое, что утверждение про весь GitHub, и завышение этого — ровно то, что вызывает острый вопрос, на который потом нечем ответить.

- Не заявлять, что DSL правил более общий, чем есть на самом деле (§4: не тьюринг-полный, осознанно).
- Не заявлять идеальную модульность — сказать про дублирование `Statistics / EntityFactory` самому (см. §4 выше), не дожидаясь, пока поймают.
- Не заявлять точную воспроизводимость запусков — `Math.random()` используется напрямую, это заявлено как ограничение.
- Не пересказывать по памяти версии ответов на §3.1–§3.4 до правок; и код, и отчёт были изменены именно потому, что те версии были неверны или неполны.

8. Финальный чек-лист перед тем, как зайти

☐

Могу без бумажки произнести: инкапсуляция vs. сокрытие информации — двумя разными фразами.

☐

Могу назвать, какие поля были `protected` до исправления, и объяснить, в чём именно была проблема `protected` (доступен из подкласса **и** из пакета, а не просто “недостаточно `private`” расплывчато).

☐

Могу запустить `Main.main()` и показать вывод про мультитипизацию вживую.

☐

Могу найти и зачитать вслух оба `catch`-блока в `MainWindow` и `catch`-блок в `RuleEditorPanel.doReload()`, не разыскивая их.

☐

Могу набрать рецепт уровня 1 (новый атрибут в словаре правил, §5.1) по памяти меньше чем за 2 минуты.

☐

Держу под рукой Part B и Part C из `theory.md` — теория DES и DSL — это два заявления “central technical differentiator” из введения самого отчёта, и самая вероятная цель глубокого допроса.